

Dental AI Assistant — Trợ lý Nha khoa Thông minh

Hệ thống **Retrieval-Augmented Generation (RAG)** chuyên sâu cho lĩnh vực nha khoa, kết hợp kho tri thức từ các bài báo y khoa chính thống (Vinmec, Pharmacy, VnExpress Sức khỏe...) với mô hình ngôn ngữ lớn (LLM) để tư vấn sức khỏe răng miệng chính xác, có trích dẫn nguồn và chống ảo giác (hallucination).

Mọi hướng dẫn vận hành, kiến trúc và câu hỏi bảo vệ được tập trung ở thư mục [docs/](#).

1. Tính năng nổi bật

Tính năng	Mô tả
Grounded Generation	Phản hồi CHỈ dựa trên bài báo trong kho dữ liệu, kèm trích dẫn nguồn rõ ràng
Query Contextualization	Hiểu ngữ cảnh hội thoại, tự động viết lại câu hỏi ngắn ("có đắt không?") thành truy vấn đầy đủ ("chi phí niềng răng tổng quan")
Hybrid Retrieval	Vector Search (FAISS) + Keyword Search (BM25) hợp nhất qua Reciprocal Rank Fusion, trọng số động theo loại câu hỏi
Multi-Query Expansion	LLM sinh 2 câu biến thể từ đồng nghĩa → tổng 3 queries, tăng recall chống Vocabulary Mismatch
Local & Cloud Hybrid	Hỗ trợ Ollama (local, miễn phí) và OpenAI (cloud) cho cả Embedding và LLM, chuyển đổi chỉ qua <code>.env</code>
Anti-Hallucination	9 quy tắc cứng trong system prompt + 5 tầng Guardrail (Query → Retrieval → Prompt → Temperature → Output)
Xác thực & Bảo mật	Đăng ký/đăng nhập, JWT, bcrypt, xác thực hai yếu tố TOTP (Google Authenticator / Authy)
Streaming UX	Server-Sent Events (SSE) — câu trả lời hiện dần từng token, disclaimer cố định do backend tự thêm

2. Kiến trúc thư mục

```
DA14_AI_Dental_Agent/  
├── api/  
│   └── main.py                # FastAPI entry point (CORS, routers, lifespan  
warmup)  
├── src/  
│   ├── agent/  
│   │   └── chatbot.py        # RAG pipeline: rewrite → (extract+expand song  
song) → search → stream  
│   ├── retriever/  
│   │   ├── engines.py        # Strategy Pattern cho Embedding (Local/OpenAI)  
│   │   ├── ingest.py         # Pipeline xây dựng FAISS index  
│   │   └── search.py         # Hybrid Search: FAISS + BM25 + RRF + Multi-
```

```

Query + Overview Boost
├── chat/
│   ├── router.py          # Endpoints /api/chat/*
│   ├── schemas.py         # Pydantic models
│   └── dependencies.py     # Lazy-init singleton chatbot
├── auth/
│   └── router.py          # Endpoints /api/auth/* (login, register, 2FA,
change password...)
│   ├── schemas.py         # Pydantic models
│   └── utils.py           # JWT, bcrypt, get_current_user
├── database/
│   ├── database.py        # SQLAlchemy engine & session
│   └── models.py          # User, ChatSession, Message
├── embedding/             # Module cũ (đã được thay bằng
src/retriever/engines.py)
├── lib/
│   └── constants.py       # Toàn bộ prompt templates + AI_TEMPERATURE
├── config.py              # Đọc .env, cấu hình tập trung
├── data/
│   ├── raw/
│   │   ├── dental_dataset.json    # Dataset gốc (762 bài, plain text)
│   │   └── dental_dataset_v2.json # Dataset nâng cấp (summary + markdown, sinh
bởi GPT-4o-mini)
│   ├── processed/
│   │   └── chunks.json            # Bản sao metadata sau ingest
│   └── vector_db/
│       ├── local/                # FAISS index cho vietnamese-sbert (768 dim)
│       └── openai/               # FAISS index cho text-embedding-3-small
(1536 dim)
├── tools/
├── upgrade_dataset.py          # Nâng cấp dataset v1 → v2 (GPT-4o-mini sinh
summary + markdown)
├── auto_pipeline.py           # Scrape + trích xuất bài viết nha khoa
├── get_links.py               # Thu thập link bài viết từ nguồn đã định
├── data/                      # Output tạm của crawler
├── links/                     # Danh sách URL đầu vào
├── frontend/
├── nextjs-app/                # Next.js 16 + React 19 + Tailwind + shadcn/ui +
Zustand
├── notebooks/
├── command/                   # Sổ tay lệnh (server.txt, client.txt,
ingest.txt, venv.txt)
├── question/                  # Bộ câu hỏi test (on-topic / off-topic)
├── docs/                      # Toàn bộ tài liệu chi tiết (xem bảng mục §6)
├── start_ollama.bat           # Script bật Ollama đã tối ưu (GPU, Flash-
Attention, Keep-Alive)
├── requirements.txt           # Python dependencies
├── .env.example               # Mẫu file môi trường
└── README.md

```

3. Yêu cầu hệ thống

Thành phần	Phiên bản	Bắt buộc?
Python	3.10+	Có
Node.js	18+	Có (chạy frontend)
MySQL / MariaDB	8.0+ / 10.6+	Có (lưu user + chat history)
Ollama	0.3+	Tùy chọn (nếu dùng engine local cho LLM)
GPU NVIDIA	≥ 4 GB VRAM	Tùy chọn (tăng tốc Ollama)
Tài khoản OpenAI	API key	Tùy chọn (nếu dùng engine openai)

4. Cài đặt & Cấu hình — Tổng quan nhanh

Xem hướng dẫn **chi tiết từng bước** (bao gồm tạo database MySQL, cài Ollama, pull model, xử lý lỗi thường gặp) tại [docs/INSTALLATION_GUIDE.md](#).

4.1. Clone & tạo môi trường Python

```
git clone https://github.com/naminc/DA14_AI_Dental_Agent.git
cd DA14_AI_Dental_Agent

python -m venv .venv
# Windows (PowerShell)
.venv\Scripts\Activate.ps1
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
```

4.2. Cấu hình biến môi trường

```
# Sao chép file mẫu → .env rồi điền giá trị
cp .env.example .env          # Linux/macOS
copy .env.example .env        # Windows
```

Các biến quan trọng (xem đầy đủ trong **.env.example**):

```
# Engine lựa chọn (thay đổi runtime chỉ qua .env)
EMBEDDING_ENGINE=local          # local | openai
LLM_ENGINE=openai              # openai | local

# OpenAI (bắt buộc nếu dùng engine openai)
OPENAI_API_KEY=sk-proj-...
OPENAI_CHAT_MODEL=gpt-4.1-mini
```

```
# Ollama (bắt buộc nếu dùng engine local cho LLM)
OLLAMA_BASE_URL=http://localhost:11434/v1
OLLAMA_CHAT_MODEL=qwen2.5:1.5b-instruct-q4_K_M

# Retrieval
TOP_K=10 # Số tài liệu đưa vào context cho LLM

# Database (MySQL)
DATABASE_URL=mysql+pymysql://root@localhost:3306/dental_agent_db

# JWT
SECRET_KEY=<sinh bằng: python -c "import secrets; print(secrets.token_hex(32))">
ACCESS_TOKEN_EXPIRE_MINUTES=1440
ALGORITHM=HS256
```

4.3. Chuẩn bị Database MySQL

```
CREATE DATABASE dental_agent_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

Các bảng (`users`, `chat_sessions`, `messages`) sẽ được **SQLAlchemy tự tạo** khi backend khởi động lần đầu (`models.Base.metadata.create_all` trong `api/main.py`).

4.4. Xây dựng FAISS index (bắt buộc, chạy 1 lần)

```
# Dùng embedding local (miễn phí, ~420 MB RAM, tải model lần đầu)
python -m src.retriever.ingest --engine local

# Hoặc dùng embedding OpenAI (trả phí, 1536 dim, chất lượng cao hơn)
python -m src.retriever.ingest --engine openai
```

Output: `data/vector_db/<engine>/faiss.index + metadata.json`.

Cả hai engine có thể tồn tại song song; `.env EMBEDDING_ENGINE` quyết định engine nào được load lúc chạy.

4.5. (Tùy chọn) Cài & bật Ollama — nếu dùng LLM local

```
# 1. Cài Ollama: https://ollama.com
# 2. Pull model đúng theo .env
ollama pull qwen2.5:1.5b-instruct-q4_K_M

# 3. Bật Ollama với cấu hình đã tối ưu (GPU + Flash-Attn + Keep-Alive 60 phút)
# Windows:
start_ollama.bat
```

```
# Linux/macOS:
OLLAMA_NUM_GPU=999 OLLAMA_FLASH_ATTENTION=1 \
OLLAMA_KEEP_ALIVE=60m OLLAMA_NUM_PARALLEL=3 \
OLLAMA_MAX_LOADED_MODELS=1 ollama serve
```

4.6. Khởi động Backend

```
uvicorn api.main:app --reload --port 8000
```

- Swagger UI: <http://127.0.0.1:8000/docs>
- Khi request đầu tiên đến, **lifespan** sẽ pre-warm **DentalChatbot** (load model ~420 MB + BM25 corpus) → các request sau chỉ còn latency LLM.

4.7. Khởi động Frontend

```
cd frontend/nextjs-app
npm install          # (hoặc pnpm install)
npm run dev          # Next.js dev server tại http://localhost:3000
```

Frontend đã có sẵn **.env.local** trỏ tới **NEXT_PUBLIC_API_URL=http://127.0.0.1:8000/api**. Nếu backend chạy ở host/port khác, sửa file **frontend/nextjs-app/.env.local** rồi restart **npm run dev**.

5. Thứ tự bật hệ thống (mỗi lần làm việc)

```
[1] Bật MySQL service
[2] (Nếu LLM_ENGINE=local) Chạy start_ollama.bat
[3] Activate Python venv
[4] uvicorn api.main:app --reload --port 8000
[5] (Tab khác) cd frontend/nextjs-app && npm run dev
[6] Mở http://localhost:3000 → Đăng ký → Đăng nhập → Chat
```

Xem checklist đầy đủ và cách kiểm tra từng bước hoạt động đúng tại **docs/RUN_GUIDE.md**.

6. Tài liệu chi tiết (thư mục **docs/**)

Tài liệu	Nội dung
README	Chỉ mục, lộ trình đọc tài liệu
INSTALLATION_GUIDE	Cài đặt từ A→Z: Python, MySQL, Ollama, pull model, lỗi thường gặp
RUN_GUIDE	Thứ tự bật hệ thống, health-check từng tầng, smoke test

Tài liệu	Nội dung
TECHNICAL_FLOW	Luồng kỹ thuật end-to-end (Rewrite → Extract + Expand → Hybrid Search → Stream)
API_ARCHITECTURE	FastAPI, Dependency Injection, SSE streaming, Pydantic schemas
FRONTEND_ARCHITECTURE	Next.js 16 / Zustand store / SSE client / 2FA flow UI
AUTH_SECURITY	JWT, bcrypt, TOTP (2FA), database schema, luồng xác thực
DATA_SCHEMA	Cấu trúc dataset v1 vs v2, pipeline nâng cấp
EMBEDDING_ENGINE	Strategy Pattern Multi-Engine (Local sbert / OpenAI)
BM25_TOKENIZATION	BM25 Okapi + Underthesea tokenization tiếng Việt
RRF_EXPLANATION	Reciprocal Rank Fusion, trọng số động, ví dụ tính tay
MULTI_QUERY_EXPANSION	LLM sinh biến thể, cross-query score merging
COSINE_TO_IP_PROOF	Chứng minh toán học Cosine $\equiv$ Inner Product khi L2-normalize
PROMPT_ENGINEERING	9 quy tắc system prompt, Temperature strategy
HALLUCINATION_GUARDRAILS	5 tầng phòng thủ chống ảo giác
DEFENSE_QA	Bộ câu hỏi & trả lời dự kiến cho hội đồng bảo vệ

7. Công nghệ sử dụng

Tầng	Công nghệ
Backend framework	FastAPI + Uvicorn
ORM	SQLAlchemy + PyMySQL
LLM	OpenAI GPT-4.1-mini (cloud) / Qwen2.5-1.5B-Instruct-Q4_K_M qua Ollama (local)
Embedding	text-embedding-3-small (1536 dim) / keepitreal/vietnamese-sbert (768 dim)
Vector DB	FAISS IndexFlatIP (Cosine $\equiv$ Inner Product trên vector L2-normalized)
Keyword Search	rank_bm25.BM25Okapi (k1=1.5, b=0.75)
NLP tiếng Việt	Underthesea word_tokenize
Frontend	Next.js 16, React 19, Tailwind CSS v4, shadcn/ui, Zustand, react-markdown
Auth	JWT (python-jose, HS256), bcrypt (passlib), TOTP (pyotp), QR code (qrcode)
Streaming	Server-Sent Events (SSE) qua StreamingResponse

8. Giấy phép & Cảnh báo y khoa

- Đây là đồ án học thuật, **không phải thiết bị y tế**.
- Mọi câu trả lời đều được backend tự gắn disclaimer: *“Thông tin chỉ mang tính tham khảo, không thay thế tư vấn trực tiếp từ bác sĩ nha khoa.”*

- Tập dữ liệu được biên soạn từ nguồn công khai, mục đích giáo dục / nghiên cứu.